



CSCE 633-700: Machine Learning

Support Vector Machines

Overview

- Lagrange Optimization
- Support Vector Machines
 - Linearly separable case
 - Non-separable case
 - Hinge Loss
 - Multi-class SVMs

Overview

- Lagrange Optimization
- Support Vector Machines
 - Linearly separable case
 - Non-separable case
 - Hinge Loss
 - Multi-class SVMs

Support Vector Machines: Linearly Separable Case

Setup for two classes

- **Input:** $\mathbf{x} \in \mathbb{R}^D$
- **Output:** $y \in \{-1, 1\}$
- **Training data:** $\mathcal{D}^{train} = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_N, y_N)\}$
- **Model:**

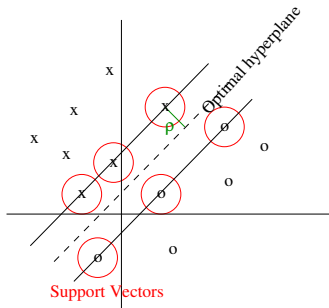
$$f(\mathbf{x}_n) = \begin{cases} 1, & \text{if } \mathbf{w}^T \mathbf{x}_n + w_0 \geq 1 \\ -1, & \text{if } \mathbf{w}^T \mathbf{x}_n + w_0 \leq -1 \end{cases}$$

We do not only require instances to be on the right side of the hyperplane (which would be $\mathbf{w}^T \mathbf{x}_n + w_0 \geq 0$)

But we also want instances some distance away $\rightarrow$ **margin**

We want to maximize this margin for **best generalization**

Optimal Hyperplane and Support Vectors



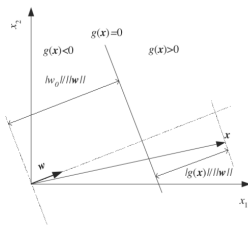
- **Margin of separation ρ :** distance between the separating hyperplane and the closest input point.
- **Support vectors:** input points closest to the separating hyperplane.

Optimal Hyperplane and Support Vectors

- The projection of a point $\mathbf{x}$ to hyperplane $\mathbf{w}$ with origin at w_0 is

$$r = \frac{g(\mathbf{x})}{\|\mathbf{w}\|} = \frac{\mathbf{w}^T \mathbf{x} + w_0}{\|\mathbf{w}\|}$$

(see Alpaydin 10.3)



Optimal Hyperplane and Support Vectors

- The projection of a point $\mathbf{x}$ to hyperplane $\mathbf{w}$ with origin at w_0 is

$$r = (\mathbf{w}^T \mathbf{x} + w_0) / \|\mathbf{w}\|$$

(see Alpaydin 10.3)

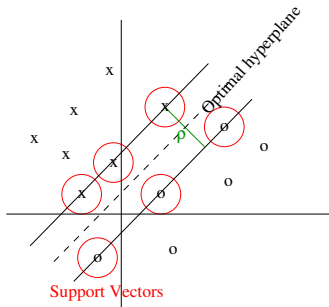
- For support vectors $\mathbf{x}_{(s)}$ on the right or left side of the hyperplane:

$$\mathbf{w}^T \mathbf{x} + w_0 = 1, \quad \mathbf{w}^T \mathbf{x} + w_0 = -1$$

- Therefore the projection of support vectors $\mathbf{x}_{(s)}$ to hyperplane $\mathbf{w}$ is

$$r = \begin{cases} 1/\|\mathbf{w}\| & \text{if } y_s = +1 \\ -1/\|\mathbf{w}\| & \text{if } y_s = -1 \end{cases}$$

Optimal Hyperplane and Support Vectors



- Margin of separation *between two classes* is

$$\rho = 2r = \frac{2}{\|\mathbf{w}\|}.$$

- Thus, maximizing the margin of separation *between two classes* is equivalent to minimizing the Euclidean norm of the weight $\mathbf{w}$!

Primal Problem: Constrained Optimization

Linearly separable case

For the training set $\mathcal{D}^{train} = \{(\mathbf{x}_n, y_n)\}_{n=1}^N$ find $\mathbf{w}$ and w_0 such that

- they minimize the *inverse* separation margin ($\frac{1}{\rho} = \frac{\|\mathbf{w}\|}{2}$) while satisfying a constraint (all examples are correctly classified):
 - Cost function: $\Phi(\mathbf{w}) = \frac{1}{2}\mathbf{w}^T \mathbf{w}$
 - Constraint: $y_n(\mathbf{w}^T \mathbf{x}_i + b) \geq 1$ for $i = 1, 2, \dots, N$

$$\min \frac{1}{2} \|\mathbf{w}\|_2^2, \text{ such that (s.t.) } y_n(\mathbf{w}^T \mathbf{x} + w_0) \geq 1, \quad n = 1, \dots, N$$

This problem can be solved using the *method of Lagrange multipliers* (see next two slides)

Support Vector Machines: Linearly separable case

$$\min \frac{1}{2} \|\mathbf{w}\|_2^2, \text{ such that (s.t.) } y_n(\mathbf{w}^T \mathbf{x}_n + w_0) \geq 1, \quad n = 1, \dots, N$$

1) Formulate Lagrangian function (primal problem)

$$L_p = \frac{1}{2} \|\mathbf{w}\|_2^2 - \sum_{n=1}^N \alpha_n [y_n(\mathbf{w}^T \mathbf{x}_n + w_0) - 1]$$

2) Minimize Lagrangian to solve for primal variables $\mathbf{w}$, w_0

$$\frac{\partial L_p}{\partial \mathbf{w}} = 0 \Rightarrow \mathbf{w} = \sum_{n=1}^N \alpha_n y_n \mathbf{x}_n$$

$$\frac{\partial L_p}{\partial w_0} = 0 \Rightarrow \sum_{n=1}^N \alpha_n y_n = 0$$

3) Substitute the primal variables $\mathbf{w}$, w_0 into the Lagrangian and express in terms of dual variables α_n

$$\begin{aligned} L_d &= \frac{1}{2} \|\mathbf{w}\|_2^2 - \mathbf{w}^T \sum_{n=1}^N \alpha_n y_n \mathbf{x}_n - w_0 \sum_{n=1}^N \alpha_n y_n + \sum_{n=1}^N \alpha_n \\ &= -\frac{1}{2} \mathbf{w}^T \mathbf{w} + \sum_{n=1}^N \alpha_n = -\frac{1}{2} \sum_{n=1}^N \sum_{m=1}^N \alpha_n \alpha_m y_n y_m \mathbf{x}_n^T \mathbf{x}_m + \sum_{n=1}^N \alpha_n \end{aligned}$$

Support Vector Machines: Linearly separable case

$$\min \frac{1}{2} \|\mathbf{w}\|_2^2, \text{ such that (s.t.) } y_n(\mathbf{w}^T \mathbf{x} + w_0) \geq 1, \quad n = 1, \dots, N$$

4) Maximize the Lagrangian with respect to dual variables (dual problem)

$$\max_{\alpha_n} L_d = \max_{\alpha_n} \left\{ -\frac{1}{2} \sum_{n=1}^N \sum_{m=1}^N \alpha_n \alpha_m y_n y_m \mathbf{x}_n^T \mathbf{x}_m + \sum_{n=1}^N \alpha_n \right\}$$

$$\text{s.t. } \sum_{n=1}^N \alpha_n y_n = 0 \text{ and } \alpha_n \geq 0, \text{ for } n = 1, \dots, N$$

- Solved numerically using quadratic optimization methods [see next slides]
- The dual depends on data size N , and particularly the number of support vectors R
- Most of the α_n will vanish with $\alpha_n = 0$, only a small percentage will have $\alpha_n > 0$
- The set of $\mathbf{x}_n$ whose $\alpha_n > 0$ are the **support vectors**

(Unconstraint) Coordinate Ascent

Unconstrained optimization problem

$$\max_{\alpha} W(\alpha_1, \alpha_2, \dots, \alpha_m)$$

We have already seen gradient descent (or ascent, if we negate the optimization function), now we consider another optimization method called coordinate ascent.

Loop until convergence

1 For $i = 1, \dots, m$

1a $\alpha_i = \arg \max_{\hat{\alpha}_i} W(\alpha_1, \dots, \hat{\alpha}_i, \dots, \alpha_m)$

- In the innermost loop, hold all variables constant except for some fixed α_i
- Re-optimize W with respect to just the parameter α_i
- When $\arg \max$ of the inner loop can be performed efficiently, coordinate ascent can be a fairly efficient algorithm

Sequential Minimal Optimization (SMO)

$$W(\alpha) = -\frac{1}{2} \sum_{n=1}^N \sum_{m=1}^N \alpha_n \alpha_m y_n y_m \mathbf{x}_n^T \mathbf{x}_m + \sum_{n=1}^N \alpha_n$$

s.t. $\sum_{n=1}^N \alpha_n y_n = 0$ and $\alpha_n \geq 0$, for $n = 1, \dots, N$

Loop until convergence

1 For $i = 1, \dots, m$

1a Select some α_i and α_j to update

1b Re-optimize W wrt α_i and α_j while holding all other α_k 's fixed

Let's assume that we optimize wrt α_1, α_2 , while $\alpha_3, \dots, \alpha_N$ are constant

- From the constraint:

$$\alpha_1 y_1 + \alpha_2 y_2 = \sum_{n=3}^N \alpha_n y_n = \zeta \rightarrow \alpha_1 = (\zeta - \alpha_2 y_2) / y_1$$

- The objective is $W(\alpha) = W((\zeta - \alpha_2 y_2) / y_1, \alpha_2, \dots, \alpha_N)$ (some quadratic function of $\alpha_2 \rightarrow$ easily solved by setting its derivative to zero)

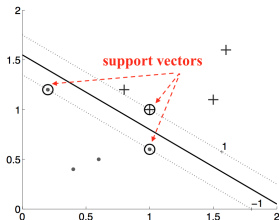
Support Vector Machines: Linearly separable case

$$\min \frac{1}{2} \|\mathbf{w}\|_2^2, \text{ such that (s.t.) } y_n(\mathbf{w}^T \mathbf{x} + w_0) \geq 1, \quad n = 1, \dots, N$$

5) Recover the solution (for the primal variables) from the dual variables

- Find $\mathbf{w}$: Substitute α_n from (4) to $\mathbf{w} = \sum_{n=1}^N \alpha_n y_n \mathbf{x}_n$
- Find w_0 :
 - From $\mathbf{w}^T \mathbf{x}_n + w_0 = y_n$, where $\mathbf{x}_n$ is a support vector, calculate $w_0 = y_n - \mathbf{w}^T \mathbf{x}_n$.
 - For numerical stability average w_0 values estimated from all support vectors.

Support Vector Machines: Linearly separable case



- Samples $\mathbf{x}_n$ for which $\alpha_n = 0$
 - majority of samples
 - lie away from the hyperplane: $y_n(\mathbf{w}^T \mathbf{x}_n + w_0) > 1$
 - have no effect on the hyperplane
- Samples $\mathbf{x}_n$ for which $\alpha_n > 0$
 - support vectors
 - lie close to the hyperplane: $y_n(\mathbf{w}^T \mathbf{x}_n + w_0) = 1$
 - determine the hyperplane

Support Vector Machines: Linearly separable case

Testing

- Testing doesn't enforce a margin, i.e. $g(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + w_0$
- Choose C_1 if $g(\mathbf{x}) > 0$, C_2 otherwise
- Function $g(\mathbf{x}_n) = \mathbf{w}^T \mathbf{x} + w_0 = \sum_{m=1}^N \alpha_m y_m \mathbf{x}_m^T \mathbf{x}_n + w_0$ does not need explicit calculation of $\mathbf{w}$
 - $g(\mathbf{x}_n)$ can be calculated from the dot product between the input vectors $\mathbf{x}_m^T \mathbf{x}_n$
 - many of α_n 's are zero, therefore computationally this is not very expensive